Journo — Trình Bày Dự Án Fullstack (Tiếng Việt)

Phần 3: Điểm Cải Thiện & Q&A Phỏng Vấn

> Tiếp theo từ Phần 2 — Giải thích sâu kỹ thuật

---

4. ĐIỂM CẢI THIỆN

4.1 Token lưu localStorage — XSS Risk

Vấn đề: `localStorage.setItem("token", token)` trong `Login.jsx` và `Register.jsx`. Bất kỳ JavaScript nào chạy trên trang (kể cả từ XSS attack qua user-generated content) đều có thể `localStorage.getItem("token")` và lấy token.

Fix:

```
// Thay vì localStorage, dùng httpOnly cookie
// Backend set cookie trong response header:
// Set-Cookie: token=abc...; HttpOnly; Secure; SameSite=Strict

// Frontend Axios config:
const api = axios.create({
  baseURL: "http://127.0.0.1:8000/api",
  withCredentials: true,  // Tự động gửi cookie theo mọi request
});
// Không cần interceptor gắn token thủ công nữa
// httpOnly cookie: JavaScript không đọc được → XSS không lấy được
```

Backend cần thêm:

```
// AuthController::login() — return cookie thay vì token trong JSON
return response()
    ->json(['user' => $user], 200)
    ->cookie('token', $plainTextToken, 60*24*7, '/', null, true, true);
    //            value,  minutes,      path, domain, secure, httpOnly
```

Tại sao tốt hơn: Loại bỏ hoàn toàn XSS token theft. Đây là security best practice cho production SPA.

4.2 Không có Protected Route — UX kém

Vấn đề: `App.jsx` không có route guard. User có thể vào `/create-post` mà không cần đăng nhập, điền xong form rồi mới biết mình chưa auth (nhận lỗi khi submit).

Fix:

```
// components/PrivateRoute.jsx
import { Navigate } from "react-router-dom";

export default function PrivateRoute({ children }) {
  const token = localStorage.getItem("token");
  // Nếu không có token: redirect về login ngay lập tức
  return token ? children : <Navigate to="/login" replace />;
}

// App.jsx
<Route path="/create-post" element={
  <PrivateRoute>
    <CreatePost />
  </PrivateRoute>
} />
```

Tại sao tốt hơn: User không điền form vô ích. UX rõ ràng hơn: thấy login page ngay.

4.3 Không có Pagination UI — User không biết còn trang nào

Vấn đề: Backend `PostController::index()` dùng `paginate(10)` và trả về `current_page`, `last_page`, `total` trong response. Nhưng `Home.jsx` chỉ đọc `res.data.data` (danh sách posts) và bỏ qua toàn bộ pagination metadata.

```
// Home.jsx hiện tại — chỉ lấy posts, bỏ pagination info
const res = await api.get("/posts");
setPosts(res.data.data);  // bỏ qua res.data.current_page, res.data.last_page
```

Fix:

```
const [page, setPage] = useState(1);
const [lastPage, setLastPage] = useState(1);

const fetchPosts = async (pageNum = 1) => {
  const res = await api.get(`/posts?page=${pageNum}`);
  setPosts(res.data.data);
  setLastPage(res.data.last_page);  // lưu total pages
};

// Render pagination controls
{page < lastPage && (
  <button onClick={() => { setPage(p => p + 1); fetchPosts(page + 1); }}>
    Load More
  </button>
)}
```

Tại sao tốt hơn: Không load tất cả posts một lúc. Performance tốt hơn khi có nhiều posts.

4.4 Slug Duplicate không được handle

Vấn đề: `PostController::store()` tạo slug từ title: `Str::slug($request->title)`. Nếu 2 bài viết có cùng title, `slug` sẽ trùng → database exception do `UNIQUE` constraint trên `posts.slug` — nhưng exception này không được catch và sẽ trả về 500 error thay vì 422 validation error.

Fix:

```
// PostController::store()
$baseSlug = Str::slug($request->title);
$slug = $baseSlug;
$counter = 1;

// Kiểm tra slug có tồn tại chưa, nếu có thì append counter
while (Post::where('slug', $slug)->exists()) {
    $slug = $baseSlug . '-' . $counter++;
}
// "my-post" → "my-post-2" → "my-post-3" ...

$post->slug = $slug;
```

Tại sao tốt hơn: Không còn 500 error. User có thể tạo nhiều bài cùng title.

4.5 Không có Rate Limiting — Login Brute Force

Vấn đề: `POST /api/login` không có rate limiting. Attacker có thể thử hàng nghìn password mà không bị chặn.

Fix:

```
// routes/api.php
Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:5,1');
    // 5 attempts per 1 minute per IP
    // Tự động trả 429 Too Many Requests khi vượt quá
```

Hoặc config global trong `bootstrap/app.php`:

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->throttleApi();  // Áp dụng rate limiting cho tất cả /api routes
})
```

Tại sao tốt hơn: Ngăn brute force attack. Laravel Throttle middleware built-in, không cần thư viện thêm.

4.6 Login trả 404 khi email không tồn tại — Information Leakage

Vấn đề: `AuthController::login()` trả `404` khi email không tìm thấy:

```
return response()->json(['message' => 'Email not found'], 404);
```

Điều này cho attacker biết email đó **không** tồn tại trong hệ thống → có thể enumerate valid emails.

Fix:

```
// Thay 404 bằng 401 với message chung chung
if (!$user || !Hash::check($request->password, $user->password)) {
    return response()->json(['message' => 'Invalid credentials'], 401);
    // Không tiết lộ email có tồn tại hay không
}
```

Tại sao tốt hơn: Attacker không thể biết email có trong hệ thống hay không. Security best practice: same error message cho cả 2 trường hợp.

4.7 Không có Error Boundary — React crash không được bắt

Vấn đề: Nếu component crash (runtime error), React 19 sẽ unmount toàn bộ app thay vì chỉ hiện lỗi ở component đó.

Fix:

```
// components/ErrorBoundary.jsx (class component — React yêu cầu)
import { Component } from "react";

export default class ErrorBoundary extends Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return <p>Something went wrong. Please refresh the page.</p>;
    }
    return this.props.children;
  }
}

// App.jsx — wrap routes
<ErrorBoundary>
```

```
  <Routes>...</Routes>
</ErrorBoundary>
```

Tại sao tốt hơn: Crash 1 component không down toàn bộ app. Graceful degradation.

4.8 Không có Test Coverage

Vấn đề: Dự án không có bất kỳ test nào — không unit test cho controllers, không integration test cho API endpoints, không component test cho React.

Fix — ưu tiên viết test theo thứ tự:

1. **API integration tests (PHPUnit)** — test quan trọng nhất:

```php
// tests/Feature/PostTest.php
public function test_unauthenticated_user_cannot_create_post()
{
    $response = $this->postJson('/api/posts', ['title' => 'Test']);
    $response->assertStatus(401);
}

public function test_user_cannot_delete_others_post()
{
    $user1 = User::factory()->create();
    $user2 = User::factory()->create();
    $post = Post::factory()->create(['user_id' => $user1->id]);

    $response = $this->actingAs($user2)->deleteJson("/api/posts/{$post->id}");
    $response->assertStatus(403);
}
```

2. **React component tests (Vitest + Testing Library)**
3. **E2E tests (Playwright)** — test full user flow

Tại sao tốt hơn: Catch regression sớm. Authorization bugs đặc biệt nguy hiểm nếu không có test.

---

5. Q&A PHỎNG VẤN

Q1: Tại sao anh chọn Laravel Sanctum thay vì tự viết JWT cho dự án này?

Tôi chọn Sanctum vì dự án này là một SPA cần token authentication, và Sanctum giải quyết đúng bài toán đó. Sanctum lưu token trong database — mỗi token là một record trong bảng `personal_access_tokens` — điều này cho phép tôi logout thật sự bằng cách xóa record đó. Với JWT thuần, token là stateless: một khi đã cấp, server không thể revoke trước khi hết hạn. Với Sanctum, khi user logout, tôi gọi `$request->user()->currentAccessToken()->delete()` và token đó vô hiệu lực ngay lập tức.

Tradeoff tôi chấp nhận là mỗi request cần một database lookup để verify token — không scalable bằng JWT khi có hàng triệu concurrent users. Nhưng với scope của dự án hiện tại, đây là sự đánh đổi hoàn toàn hợp lý.

Q2: JWT hoạt động như thế nào? Và Sanctum token khác JWT như thế nào?

JWT gồm 3 phần: `header.payload.signature`. Header chứa algorithm (HS256), payload chứa claims như `user_id`, `exp` (expiry time), signature là HMAC của `header.payload` với secret key. Server verify bằng cách re-compute signature và so sánh — không cần database. Đây là điểm mạnh (stateless, scalable) nhưng cũng là điểm yếu (không revoke được).

Sanctum token hoàn toàn khác: nó chỉ là một random string opaque. Server nhận token, hash nó bằng SHA-256, tìm hash đó trong database. Nếu tìm thấy thì authenticated. Không có payload, không có expiry mặc định (theo config `sanctum.expiration = null`). Tôi phải tự implement logout bằng cách xóa record.

Q3: Tại sao anh lưu token trong localStorage thay vì httpOnly cookie? Và nhược điểm là gì?

Tôi chọn localStorage vì đơn giản nhất cho development phase — không cần xử lý CSRF, không cần cấu hình `withCredentials` trên Axios. Axios interceptor đọc token từ localStorage và gắn vào `Authorization: Bearer` header — pattern quen thuộc và dễ debug.

Nhược điểm lớn nhất là XSS vulnerability. Nếu attacker inject được JavaScript vào trang — dù chỉ một script tag trong content của bài viết — họ có thể `localStorage.getItem('token')` và lấy token của user. httpOnly cookie thì JavaScript không đọc được, kể cả code inject từ XSS.

Trong production tôi sẽ migrate sang httpOnly cookie với `SameSite=Strict` và implement CSRF protection. Đây là trade-off tôi nhận thức rõ và chấp nhận tạm thời cho dự án học tập này.

Q4: Eager loading là gì? Anh dùng nó ở đâu và tại sao?

Eager loading là technique load tất cả related data trong một vài queries thay vì load lazy (N+1 queries). Trong `PostController::index()` tôi viết `Post::with('author', 'category', 'tags')`. Câu lệnh này thực hiện 4 queries: một cho posts, một cho users (author), một cho categories, một cho tags. Kết quả được join ở application level.

Nếu không dùng eager loading, khi render 10 posts, mỗi lần access `$post->author->name` sẽ trigger thêm 1 query. 10 posts = 10 thêm queries cho author, 10 cho category = tổng 21+ queries thay vì 4. Với 100 posts thì đó là vấn đề nghiêm trọng về performance.

Q5: Anh handle authorization như thế nào? Backend làm gì để ngăn user A xóa bài của user B?

Tôi implement authorization check trực tiếp trong controller, trước khi thực hiện action. Pattern cụ thể trong `PostController::destroy()`: sau khi tìm thấy post, tôi so sánh `$post->user_id` với `$request->user()->id`. `$request->user()` được lấy từ Sanctum — nó là user đã xác thực, không thể giả mạo từ request body.

Nếu hai giá trị không khớp, tôi trả về 403 Forbidden ngay lập tức. Pattern này giống nhau ở cả `update()` và `destroy()`. Điểm tôi muốn cải thiện là dùng Laravel Policies — tách authorization logic ra khỏi controller thành một class riêng `PostPolicy`, giúp code sạch hơn và dễ test hơn.

Q6: Nhược điểm lớn nhất của architecture hiện tại là gì?

Có ba điểm tôi nghĩ là nhược điểm đáng kể nhất.

Thứ nhất, token lưu localStorage — như đã nói, XSS risk thực sự.

Thứ hai, không có Protected Route ở frontend. User có thể vào `/create-post` mà chưa login, điền form xong mới bị lỗi 401 — UX tệ. Fix bằng một `PrivateRoute` component đơn giản.

Thứ ba, không có refresh token. Sanctum tokens mặc định không expire, nhưng nếu tôi set `sanctum.expiration`, user sẽ bị logout đột ngột giữa session mà không có cơ chế tự renew token silently.

Q7: Nếu scale lên 10× user, anh sẽ thay đổi gì?

Với 10× user, bottleneck đầu tiên sẽ là database. Tôi sẽ thêm index cho những query thường gặp nhất: `posts(status, published_at)` cho home page query, `posts(user_id)` cho profile page. Hiện tại Eloquent thêm index tự động chỉ cho foreign keys.

Tiếp theo, implement caching cho public data — danh sách posts, categories — không cần query DB mỗi request. Laravel có Redis cache built-in. Token verification cũng có thể cache.

Thứ ba, migrate sang httpOnly cookie và add rate limiting cho auth endpoints để chặn brute force.

Cuối cùng, nếu scale hơn nữa, tách Sanctum token lookup thành Redis store thay vì MySQL — giảm DB load đáng kể.

Q8: Comment threaded hoạt động như thế nào trong database của anh?

Tôi dùng adjacency list pattern: bảng `comments` có column `parent_id` tự reference về chính bảng đó. `parent_id = NULL` nghĩa là top-level comment. `parent_id = 5` nghĩa là reply cho comment id=5.

Trong `CommentController::index()`, tôi query chỉ top-level comments (`whereNull('parent_id')`) và eager load replies: `Comment::with('user', 'replies.user')`. `replies.user` là nested eager loading — load replies và user của mỗi reply trong 2 thêm queries thay vì N queries.

Nhược điểm của adjacency list: nếu muốn lấy tất cả descendants (replies của replies của replies), cần recursive query hoặc multiple queries. Với nested reply 2 cấp thì ổn, nhưng nếu muốn unlimited depth, cần dùng nested set model hoặc closure table.

Q9: Có vấn đề gì với code hiện tại mà anh muốn sửa ngay không?

Có, và tôi muốn thành thật về điều này. Vấn đề tôi lo nhất là slug duplicate. Khi 2 bài viết có cùng title, `Str::slug()` sẽ tạo ra 2 slug giống nhau, nhưng database có UNIQUE constraint trên `posts.slug` — kết quả là 500 error không được handle gracefully. Fix đơn giản là loop thêm counter: "my-post", "my-post-2", v.v.

Vấn đề thứ hai là login endpoint trả `404` với message "Email not found" — điều này leak thông tin: attacker biết email đó không tồn tại. Cần thay thành `401` với message chung "Invalid credentials" cho cả 2 trường hợp.

Q10: Khó khăn lớn nhất khi làm project này là gì?

Khó khăn kỹ thuật lớn nhất là hiểu Sanctum token flow lần đầu. Tôi bị nhầm giữa `plainTextToken` và token hash được lưu trong DB — cứ tưởng token gửi về cho client là token đã hash, nhưng thực ra ngược lại: plaintext gửi cho client, hash lưu trong DB. Phần `id|randomString` trong token format cũng mất thời gian debug — `id` là `tokenable_id` trong `personal_access_tokens`, dùng để tìm record trước khi verify hash.

Khó khăn thứ hai là CORS giữa Vite dev server (port 5173) và Laravel (port 8000). Lúc đầu tôi cứ nghĩ CORS là lỗi của frontend, nhưng thực ra hoàn toàn là server-side config. Sau khi hiểu rõ CORS là browser security policy, server phải response với `Access-Control-Allow-Origin` header, mọi thứ chạy trơn tru hơn.

Kết thúc series trình bày Tiếng Việt.
Xem thêm:
- presentation_en_1_overview_architecture.md — English version
- presentation_fr_1_overview_architecture.md — French version